ZKPriv: Zero-Knowledge Proofs That LLM Responses Contain No Prohibited Personal Data

Huzaif Khan*, Ghazal Mahdian†, Ali Jalooli‡
Department of Computer Science, California State University, Dominguez Hills, Carson, CA 90747, USA
*hkhan17@toromail.csudh.edu, †gmahdian1@toromail.csudh.edu, ‡ajalooli@csudh.edu

***Abstract*—Cloud-hosted large language models (LLMs) can reproduce personally identifiable information (PII) from their context or training data, and a provider can serve such a response to one user while appearing compliant to auditors. Zero-knowledge (ZK) systems for LLMs prove that a model computed an output correctly, but not what the output contains. We present ZKPRIV, a system in which every response carries a ZK proof that the committed response bytes contain no structured PII forbidden by the provider's registered policy, without revealing the prompt, the response, or the model. Unlike designs in which the provider supplies PII labels as a private input, ZKPRIV computes detection inside the circuit: each detector is compiled to a minimized search automaton, and the proof shows that no automaton reaches an accepting state anywhere in the response. A left-to-right automaton tracks the Luhn checksum so that payment cards are recognized without look-ahead. The circuit also binds the response length and padding, an optional ASCII guard, a Poseidon2 commitment, and a versioned policy hash. Implemented in Noir with zero-knowledge UltraHonk on an Apple M4 Pro CPU, a 256-byte response checked by six detectors compiles to 177,533 gates, proves in 0.67 s, and verifies in 10 ms with a 14.3 KiB proof. Names and places, which no pattern captures, are handled by a second tier: an enclave runs named-entity recognition and signs the labels over the response commitment, and the circuit verifies the signature and enforces the policy on them, adding 38,508 gates and proving in 0.77 s. The circuit agrees with independent reference detectors on all 300 test inputs, rejects all 4 label-forgery attacks we tried, and, compared with Presidio on the same 1,000 records, detects more payment cards while refusing far fewer texts whose labelled PII has been removed (0.0% vs. 23.0% on synthetic data).**

***Index Terms*—Large language models, zero-knowledge proofs, privacy compliance, PII detection, finite automata, verifiable computation**

I. INTRODUCTION

Cloud LLMs are increasingly used in healthcare, finance and customer service, where responses may only be released if they do not disclose regulated personal data. LLMs reproduce PII from their prompts, retrieved documents and training data [1]. A provider can filter outputs, but a filter offers no evidence to anyone outside the provider: a provider that disables it for one user, or for one request, looks the same to an auditor as one that never did.

Privacy-enhancing technologies for LLM inference—MPC, FHE, trusted execution environments (TEEs) and prompt obfuscation [2]—hide the prompt or the response from the provider. They do not certify what the response contains. Zero-knowledge systems for LLMs, such as zkLLM [3] and zkGPT [4], prove that a committed output was computed by a committed model; they say nothing about whether that output contains a Social Security number. Output guardrails [5], [6] check content but are not verifiable.

The previous version of this work proved compliance with respect to sensitivity labels that the provider computed outside the proof and supplied as a private witness. A dishonest prover can set every label to "not sensitive", so such a proof certifies only what the provider claims. This paper removes that gap for structured PII, whose labels are *computed inside the circuit* from the committed response bytes, and narrows it for names and places, whose labels must be signed by an attested enclave over the same commitment.

Contributions.

1) **In-circuit absence proofs for structured PII.** Detectors for SSNs, e-mail addresses, US phone numbers, IPv4 addresses, numeric dates and payment cards are compiled to minimized search automata with absorbing accept states, so "the response contains no match" reduces to a single check on the final state. A dedicated automaton tracks two Luhn sums left to right, recognizing valid card numbers without look-ahead.
2) **A policy-bound response receipt.** The proof binds a Poseidon2 [7] commitment to the response, its exact length and zero padding, the request identifier, and the hash of a versioned policy that selects detectors. The user who receives a response checks it against the commitment, so a provider cannot certify one response and deliver another.
3) **A second tier for semantic PII.** Names and places are labelled by named-entity recognition inside an enclave, which signs the labels over the response commitment; the circuit verifies the secp256k1 signature and enforces the policy on the signed labels. The structured-PII guarantee holds without trusting the enclave; only the semantic tier depends on it.
4) **An implementation and measurement.** We implement ZKPRIV in Noir [8] with zero-knowledge UltraHonk [9] and report measured circuit size, proving and verification time, memory and proof size on commodity hardware, circuit-to-reference agreement, resistance to label forgery, and detection on two PII corpora, compared with Presidio, and on real LLM outputs. All code, data and scripts that produce the numbers in this paper are released.

II. RELATED WORK

Private and verifiable LLM inference. MPC [10], [11], FHE [12], [13] and TEE-based inference [14] protect prompts

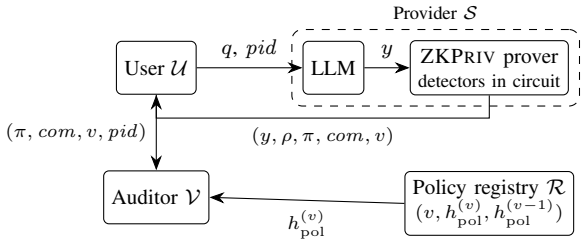


Fig. 1. ZKPRIV workflow. The provider proves that the committed response contains no structured PII forbidden by policy version v. The user checks the commitment opening and the proof; any auditor can verify a forwarded receipt without seeing the response.

and responses from the provider; prompt-side techniques [15], [16] reduce what is sent. zkLLM [3] and zkGPT [4] prove inference correctness, and FairProof [17] certifies model fairness. None proves a property of an individual response's content. ZKPRIV is complementary: its commitment can be the output commitment of a correctness proof.

ZK proofs over text. Reef [18] proves that a committed document matches, or does not match, a regular expression, and zk-regex [19] and Londhe *et al.* [20] prove regex matches for e-mail verification. Path-witness constructions prove that a match *exists*; they cannot prove absence. ZKPRIV applies a forced, whole-string automaton run, as in Reef's non-match case, to LLM responses, and adds what output compliance needs: detectors with checksum validation, length and padding binding, policy selection with versioning, and per-request binding. Zero-Knowledge Middleboxes [21] prove that encrypted traffic complies with a policy; ZKPRIV transfers that model to LLM responses.

Governance and filtering. Blockchain audit logs [22], [23] record that a check occurred but not that a specific response passed it. Guardrails [5], [6], [24] and PII detectors such as Presidio [25] and our earlier edge framework SenseCLLM [26] filter text but produce no verifiable evidence.

III. System Model

Principals. A user $\mathcal{U}$ sends prompt q with a fresh request identifier pid to a provider $\mathcal{S}$ that runs an LLM. $\mathcal{S}$ publishes policies in an append-only, hash-chained registry $\mathcal{R}$ of records $(v, h_{\text{pol}}^{(v)}, h_{\text{pol}}^{(v-1)}, \tau_v)$. An auditor $\mathcal{V}$ (a regulator or the user) holds the public verification key and reads $\mathcal{R}$ (Fig. 1).

Protocol. For response y (UTF-8 bytes, $|y| \le N$), $\mathcal{S}$ samples a blinding value ρ and computes

$$\begin{aligned} com &= H\big(\text{pack}(y \,\|\, 0^{N-|y|}),\ |y|,\ \rho,\ pid\big), \\ h_{\text{pol}} &= H(m, v), \end{aligned} \tag{1}$$

where H is a Poseidon2 sponge over BN254 and m is the policy mask. $\mathcal{S}$ proves the statement of Section IV and returns (y, ρ, π, com, v). $\mathcal{U}$ recomputes com from the received y and ρ, checks that h_{pol} equals the registry entry for v and that v is current, and verifies π. A receipt (π, com, v, pid) can be forwarded to $\mathcal{V}$, who verifies it without y.

Threat model. $\mathcal{S}$ is malicious: it may run any model, alter or skip filters, and try to certify a response other than the one delivered. The user is honest for its own checks. We consider:

- *A1, false certification:* delivering y^* containing PII with a valid receipt. Prevented by soundness of the proof together with the user's commitment check, which requires a Poseidon2 collision to open com to a different string.
- *A2, policy downgrade:* proving against a weaker, older policy. Prevented by binding h_{pol} and v to the proof and checking v against the head of $\mathcal{R}$.
- *A3, replay:* reusing a receipt for another request. Prevented by binding pid into com.
- *A4, hiding past the end:* placing PII after the declared length. Prevented because the circuit forces every byte beyond $|y|$ to zero and binds $|y|$.
- *A5, label forgery (tier 2):* claiming that a response contains no names, or reusing labels from another response. Prevented because the labels must carry the enclave's signature over a digest that includes com.

Trust. Tier 1 (structured PII) needs no trust in a TEE or a classifier binary: the detectors are public circuit code. Tier 2 (names, places, other identifiers recognized by NER) trusts that the enclave's signing key is bound by remote attestation to the published NER code; a compromised enclave can under-label, but cannot weaken tier 1. Out of scope are PII that neither tier expresses (for example "john at example dot com"), cross-query aggregation, and side channels.

IV. Proving the Absence of Structured PII

A. From Detectors to Search Automata

Each detector D_k is a regular expression over bytes with explicit boundaries: an SSN must not be preceded or followed by an alphanumeric byte, so "1234-56-78901" is not reported. We write the start and end of the text as a virtual byte β. To prove that D_k occurs nowhere in y, we compile it to a DFA for $\Sigma^* D_k$ by Thompson construction and subset construction [27], re-injecting the start state at every step so that matches beginning at any position, including overlapping ones, are tracked. We then make accepting states absorbing and minimize the DFA. After these steps

$$D_k \text{ occurs in } \beta y \beta \iff \delta_k^*(s_0, \beta y \beta) = a_k, \tag{2}$$

where a_k is the unique accepting state left by minimization. A witness that "skips" a match is impossible: every transition is fixed by the table, so the circuit only checks the final state.

To shrink the tables we partition the 256 byte values into classes with identical transition columns. A step costs two table lookups: byte to class, then (state, class) to the next state.

B. Payment Cards and the Luhn Check

Card numbers are 12–19 digits (ISO/IEC 7812; Maestro cards may have 12), optionally grouped by single spaces or hyphens, and valid numbers satisfy the Luhn checksum [28]. Luhn doubles every second digit *counted from the right*, which a left-to-right automaton cannot know in advance. We therefore track both possibilities. After digits $d_1 \dots d_k$, let S_0 be the

checksum if d_k is the last digit and S_1 the checksum if d_k is doubled. Reading digit d gives

$$S_0' = d + S_1, \qquad S_1' = \mathrm{dbl}(d) + S_0 \pmod{10}, \quad (3)$$

with $\mathrm{dbl}(d) = 2d - 9[d \geq 5]$. A run ending at a non-alphanumeric byte with $12 \leq k \leq 19$ and $S_0 \equiv 0$ is a card. The automaton has states $(k, S_0, S_1, \text{after-separator})$ plus idle, in-word and too-long states, which minimizes to a few thousand states (Table I).

C. The Circuit

The private inputs are the padded bytes $y_0 \dots y_{N-1}$, length ℓ, blinding ρ, policy mask m and version v; pid is public, and (com, h_{pol}) are public outputs. The circuit enforces:

P1 *Length and padding:* $\ell \leq N$; $y_i \neq 0$ for $i < \ell$ and $y_i = 0$ for $i \geq \ell$.

P2 *ASCII guard (policy bit 6):* $y_i < 128$ for all i, which rejects Unicode digits and look-alike characters that byte-level detectors would otherwise miss.

P3 *Detectors (policy bits 0–5):* for each enabled D_k, $\delta_k^*(s_0, \beta y_{0..\ell-1}\beta) \neq a_k$; the transition at position i is applied only when $i < \ell$.

P4 *Commitment and policy:* com and h_{pol} are computed as in (1), packing 31 bytes per field element.

A response that violates the policy has no satisfying witness, so the prover fails before producing a proof.

D. Tier 2: Enclave-Signed Semantic Labels

Patterns cannot recognize names or places. In tier 2 the enclave that runs the LLM also runs named-entity recognition (Presidio [25] with spaCy) and sets a label mask σ with one bit each for PERSON, LOCATION, NRP and other identifiers (passport, driver licence, bank account). It computes $d = H(com, \sigma)$ and signs d with ECDSA over secp256k1, using a key bound to the enclave by remote attestation. The circuit additionally takes σ and the signature as private inputs and the enclave public key as a public input, and enforces

P5 *Signed labels:* $\mathrm{Verify}_{pk}(\mathrm{sig}, H(com, \sigma)) = 1$ and $\sigma \wedge m_{\mathrm{sem}} = 0$, where m_{sem} is the semantic part of the policy mask.

Because the digest contains com, labels signed for one response cannot be reused for another, and changing σ invalidates the signature. The enclave computes d with the same Poseidon2 code as the circuit, so the two cannot disagree.

Security. The relation is $\mathcal{L} = \{(pid, com, h_{\mathrm{pol}}) : \exists(y, \ell, \rho, m, v)$ satisfying P1–P4 (and P5 in tier 2)$\}$. By knowledge soundness of UltraHonk, a verifying proof yields such a witness. By (2) and the correctness of compilation, which we test differentially (Section V), the extracted y contains no match of any enabled detector. By collision resistance of Poseidon2, the user's opening (y, ρ) of com equals the extracted y, which gives A1; binding h_{pol}, v and pid gives A2 and A3, P1 gives A4, and unforgeability of ECDSA together with the commitment inside the signed digest gives A5. Zero knowledge of the UltraHonk ZK variant hides y, ρ and m from $\mathcal{V}$. The guarantee is relative to the detector language: it says that no *expressible* PII is present, not that no PII is present.

TABLE I
CIRCUIT SIZE AT $N = 256$: BASE CIRCUIT AND GATES ADDED BY EACH DETECTOR ALONE.

Component	UltraHonk gates
Base: P1, P2, P4 (no detectors)	7,388
+ SSN	5,467
+ E-mail	5,389
+ US phone	6,412
+ IPv4	5,754
+ Numeric date	5,824
+ Payment card (Luhn)	144,169
All six detectors	177,533

TABLE II
COST BY RESPONSE CAPACITY N, ALL SIX DETECTORS; MEDIAN OF 10 RUNS ON AN APPLE M4 PRO CPU. PROOF SIZE IS 14.3 KIB THROUGHOUT.

N (B)	Gates	Wit. (s)	Prove (s)	Verify (ms)	MiB
64	155,575	0.20	0.58	10	335
128	162,919	0.21	0.59	10	344
256	177,533	0.22	0.67	10	366
512	206,835	0.26	0.74	10	445
1024	265,439	0.34	0.95	10	648

V. EVALUATION

Setup. We implement the detector compiler in Python and generate the Noir circuit from it (Noir 1.0.0-beta.22, Barretenberg 5.0.0-nightly.20260522, zero-knowledge UltraHonk with a Poseidon2 transcript). Measurements use one Apple M4 Pro with 24 GB of memory, CPU only. Each configuration is proved 10 times; we report medians. The verification key is computed once per circuit and excluded from proving time.

A. Correctness of the Circuit

Each compiled DFA was tested against an independent implementation written with Python's `re` module (and a direct Luhn implementation for cards) on 200,000 random strings seeded with PII-like fragments: all 1.2 million detector decisions agree. The compiled Noir circuit, run through witness generation, refuses exactly the inputs the reference flags on 300/300 test inputs. Differential testing exposed one boundary bug during development (an e-mail address at the end of a sentence was missed), which we fixed before all reported measurements.

B. Cost

Table I breaks down the circuit at $N = 256$. Five detectors each add about 5.5–6.5 k gates. The card detector adds 144,169, almost all of it the one-time cost of committing its transition table, whose Luhn state space makes it two orders of magnitude larger than the others. Table II shows that cost grows slowly with the response capacity: from $N = 64$ to $N = 1024$ the circuit grows by about 114 gates per byte for all six detectors together, and proving time rises from 0.58 s to 0.95 s.

TABLE III
PER-VALUE RECALL: ZKPRIV (IN-CIRCUIT DETECTORS AND, FOR NAMES AND PLACES, TIER-2 NER) VS. PRESIDIO ON THE SAME RECORDS. OTHER ANNOTATED VALUES IN EACH RECORD ARE MASKED SO THAT EACH VALUE IS TESTED ALONE.

	ai4privacy (500)		Synthetic US (500)	
Type	ZKPRIV	Presidio	ZKPRIV	Presidio
SSN	0.727	0.703	1.000	1.000
E-mail	0.991	0.935	0.821	0.821
US phone	0.333	0.399	0.843	0.860
IPv4	—	—	1.000	1.000
Date	0.390	0.825	0.671	1.000
Card	0.216	0.108	1.000	0.720
Name (tier 2)	0.385		0.897	
Place (tier 2)	0.210		—	
Refused, has PII	85.0%	97.5%	100.0%	99.6%
Refused, PII removed	22.6%	34.8%	0.0%	23.0%

Verification stays at about 10 ms and proofs are 14.3 KiB at every size. A 1 KB response is therefore certified in under a second on a laptop CPU, which is small compared with the time an LLM takes to generate it. For comparison, published correctness proofs for GPT-2 inference take tens of seconds and produce proofs of about 100 KB [4]. ZKPRIV certifies a different property and can be composed with such a proof through the shared output commitment.

C. Detection

Table III reports detection on 500 English records of `ai4privacy/pii-masking-400k` [29] and on a 500-record synthetic corpus of US-format PII in four domains. On US formats the detectors find every SSN and IPv4 address; missed e-mail addresses are deliberately obfuscated ("name AT host DOT com"), and missed phone numbers are UK numbers outside the US pattern. On ai4privacy most misses are formats the detectors do not express: non-US phone numbers, ten-digit "social numbers", written dates such as "May/58", and randomly generated card numbers that fail the Luhn check. After every annotated value is removed, 22.6% of ai4privacy records still trigger a detector; inspection shows that most of these contain unannotated identifiers (passport, account and card-like numbers) or non-birth dates, which a strict policy blocks by design. The policy mask allows deployments to disable the date detector where dates are benign.

Comparison with Presidio. Presidio [25] (spaCy `en_core_web_sm`, default threshold) is the detector a provider would most likely run outside a proof. On the same records and protocol, ZKPRIV's in-circuit detectors match it on SSNs and e-mail and find more payment cards (1.000 vs. 0.720 on synthetic data), because the Luhn automaton accepts 12–19-digit numbers with separators. Presidio is better on dates, which it also recognizes in written form, and on international phone numbers. Its main cost is over-blocking: after all labelled PII is removed it still flags 23.0% of synthetic and 34.8% of ai4privacy records, against 0.0% and 22.6% for ZKPRIV. Name and place recall, which tier 2 obtains from the same NER model, is high on clean synthetic text (0.897) but low on ai4privacy (0.385 and 0.210), whose "names" are often username-like strings; a stronger NER model can replace the small spaCy model without changing the circuit.

D. Tier 2: Cost and Forgery Resistance

Adding signed semantic labels increases the circuit at $N = 256$ from 177,533 to 216,041 gates (38,508 for ECDSA verification and the label digest), proving time from 0.67 s to 0.77 s and verification to 11 ms, with the same proof size. In the enclave, NER takes a median 7 ms per response once the model is loaded and signing 0.9 ms. Our enclave is simulated: the code and keys are real, but we do not reproduce hardware attestation. We tested 4 forgery attempts—zeroing the labels, flipping one signature bit, using another enclave's key, and reusing a valid attestation for a different response—and the circuit refused all 4.

E. Real LLM Outputs

We gave llama3.2:3b (Ollama, temperature 0) a synthetic customer record and asked it either to list the record's identifiers or to draft a reply confirming the contact details on file, plus one general question without a record, for 180 responses in total (median 201 bytes, maximum 858). 12 responses reproduced at least one identifier verbatim; ZKPRIV refused 12 of them, so no receipt could be produced. It refused 0 of 60 general answers. The model declined to list identifiers in most cases, which is why few responses contain PII. Counted per value, the detectors caught every reproduced SSN (4/4), IPv4 address (12/12) and card number (6/6); the uncaught values were written-out dates, a UK phone number and e-mail addresses the record itself spelled as "name AT host DOT com". Each of those responses was still refused because it contained another detectable identifier. Inspecting these outputs also revealed that our first card detector required 13 digits and missed 12-digit Maestro numbers; we widened the range to 12–19 and re-ran all experiments.

F. Limitations

ZKPRIV certifies the absence of PII that its two tiers can express. Tier 1 is exact for its patterns but not for paraphrase: adversarial spellings such as "one two three" defeat byte-level detectors, and the ASCII guard removes only Unicode confusables. Tier 2 is only as good as the NER model and requires trusting enclave attestation, which our prototype simulates. The card automaton merges a card with an adjacent digit group joined by a single separator. Responses longer than N require a larger circuit or chunked proofs, and per-response checks do not bound leakage across many queries. Finally, the guarantee depends on the correctness of the detector compiler, which we test differentially rather than verify formally.

VI. CONCLUSION

ZKPRIV lets an LLM provider attach to each response a zero-knowledge proof that the response contains no structured PII forbidden by its registered policy, with detection computed inside the circuit rather than asserted by the provider, and

extends the guarantee to names and places through enclave-signed labels. On a laptop CPU, a 256-byte response with six detectors is proved in 0.67 s and verified in 10 ms. With signed semantic labels, proving takes 0.77 s. Future work will run the enclave on attested hardware, aggregate receipts across queries, and compose ZKPRIV with inference-correctness proofs.

REFERENCES

[1] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, 2021, pp. 2633–2650.

[2] D. Andreoletti, A. Rudi, E. Carpanzano, F. Lelli, and T. Leidi, "Privacy-preserving llm inference in practice: A comparative survey of techniques, trade-offs, and deployability," 2026, preprint.

[3] H. Sun, J. Li, and H. Zhang, "zkllm: Zero knowledge proofs for large language models," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM, 2024, pp. 4405–4419.

[4] W. Qu, Y. Sun, X. Liu, T. Lu, Y. Guo, K. Chen, and J. Zhang, "zkgpt: An efficient non-interactive zero-knowledge proof framework for llm inference," in *Proceedings of the 34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2045–2063.

[5] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2023, pp. 431–445.

[6] Meta AI, "Llama Guard 2: Safety-oriented LLM for human-AI conversations," arXiv preprint arXiv:2404.08893, 2024.

[7] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A faster version of the poseidon hash function," in *Progress in Cryptology – AFRICACRYPT 2023*, 2023.

[8] Noir contributors, "Noir: A domain-specific language for zero-knowledge proofs," https://noir-lang.org, 2026, version 1.0.0-beta.22.

[9] Aztec Labs, "Barretenberg: UltraHonk proving system," https://github.com/AztecProtocol/aztec-packages/tree/next/barretenberg, 2026, version 5.0.0-nightly.20260522.

[10] Y. Dong *et al.*, "Puma: Secure inference of llama-7b in five minutes," 2023.

[11] K. Gupta *et al.*, "Sigma: Secure gpt inference with function secret sharing," 2023.

[12] L. de Castro *et al.*, "Encryptedllm: Privacy-preserving large language model inference via gpu-accelerated fully homomorphic encryption," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[13] D. Park, E. Lee, and J.-W. Lee, "Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 11 090–11 111.

[14] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, "Confidential llm inference: Performance and cost across cpu and gpu tees," 2025.

[15] M. Tong *et al.*, "Inferdpt: Privacy-preserving inference for closed-box large language models," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 5, pp. 4625–4640, 2025.

[16] Z. Zeng *et al.*, "Privacyrestore: Privacy-preserving inference in large language models via privacy removal and restoration," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 10 821–10 855.

[17] C. Yadav, A. Roy Chowdhury, D. Boneh, and K. Chaudhuri, "FairProof: Confidential and certifiable fairness for neural networks," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, 2024.

[18] S. Angel, E. Ioannidis, E. Margolin, S. Setty, and J. Woods, "Reef: Fast succinct non-interactive zero-knowledge regex proofs," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[19] ZK Email, "zk-regex: Regex verification circuits," https://github.com/zkemail/zk-regex, 2025.

[20] Londhe *et al.*, "ZK proofs of generalized regex matching," *Proceedings on Privacy Enhancing Technologies*, 2026, cryptology ePrint Archive 2026/1284.

[21] P. Grubbs, A. Arun, Y. Zhang, J. Bonneau, and M. Walfish, "Zero-knowledge middleboxes," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[22] S. Jan, H. A. Razzaqi, A. Akarma, and M. R. Belgaum, "A blockchain-monitored agentic ai architecture for trusted perception–reasoning–action pipelines," 2025, presented at the IEEE International Conference on Computing and Applications (ICCA 2025), Bahrain; arXiv:2512.20985.

[23] D. W. H. Ameen, "Ai-driven secure audit trails for financial compliance using immutable blockchain logs," in *Proceedings of the 3rd International Conference on Cyber Resilience (ICCR 2025)*. IEEE, 2025.

[24] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 9004–9017.

[25] Microsoft Corporation, "Presidio: Data protection and PII anonymization SDK," GitHub, 2023, available: https://github.com/microsoft/presidio.

[26] H. Khan, G. Mahdian, and A. Jalooli, "SenseCLLM: A three-agent edge-resident framework for adaptive PII detection and selective anonymization in IoT-to-LLM pipelines," in *IEEE World AI IoT Congress (AIIoT)*, 2026.

[27] M. O. Rabin and D. Scott, "Finite automata and their decision problems," *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.

[28] H. P. Luhn, "Computer for verifying numbers," U.S. Patent 2,950,048, 1960.

[29] ai4privacy, "pii-masking-400k: Large-scale annotated PII masking dataset," Hugging Face Datasets, 2024, available: https://huggingface.co/datasets/ai4privacy/pii-masking-400k.